Karatina University
PHY 401 Project Assignment

Isothermal and Adiabatic Compression in a Laboratory Air Supply

In this PHY 401 Thermal Physics II design scenario, a Karatina University laboratory is assessing a proposed dry air compressor for pneumatic instruments at 8 bar absolute. A single stage unit may run too hot, so a two stage system with an intercooler is under consideration. Use Python to compare reversible isothermal, reversible adiabatic and practical adiabatic compression. Evaluate the discharge temperature and motor limits, then recommend a configuration. The numerical data are supplied for this assignment and are not measured specifications of existing Karatina University equipment.

Learning outcomes: Apply the first and second laws to steady flow compression; distinguish isothermal, isentropic and irreversible adiabatic paths; implement and verify a numerical model; and defend a practical design choice using uncertainty and equipment constraints.

Operating specification

Parameter	Base case	Range or constraint
Air inlet pressure p1	100 kPa absolute	90–110 kPa
Required delivery pressure p3	800 kPa absolute	600–1,000 kPa
First stage inlet temperature T1	298 K	288–308 K
Mass flow rate m_dot	0.060 kg/s	0.040–0.080 kg/s
Intercooler outlet temperature T3	303 K	At least 5 K above coolant temperature
Final aftercooler outlet temperature	310 K	Delivery condition
Coolant reference temperature Tc	293 K	Assume a large constant temperature reservoir
Air gas constant R	0.287 kJ/(kg K)	Ideal gas approximation
Constant pressure heat capacity cp	1.005 kJ/(kg K)	Use constant cp in core model
Heat capacity ratio gamma	1.400	Verify gamma = cp/(cp − R) approximately
Isentropic efficiency per stage eta_s	0.78	0.70–0.85
Motor and drive efficiency eta_m	0.90	0.85–0.95
Maximum air discharge temperature	500 K	Applies at outlet of each compression stage
Available motor rating	22 kW	Electrical input must not exceed rating
Operating time	2,000 h/year	Use for annual electricity only

All pressures are absolute. Neglect changes in kinetic and potential energy, pressure losses in coolers, and heat transfer through compressor casings. Treat each compressor as steady flow and adiabatic. For the two stage case, choose intermediate pressure p2 between p1 and p3; the intercooler restores air to T3 before stage two.

Thermodynamic model and tasks

1 Derive reference limits

Derive the reversible isothermal specific work w_iso = R T1 ln(p3/p1) and cooling magnitude q_iso,out = w_iso. Derive the reversible adiabatic outlet temperature T_out,s,one = T1(p3/p1)^[(gamma − 1)/gamma] and specific work w_s = cp(T_out,s,one − T1). Explain the boundary conditions for each ideal process and why isothermal work is a lower bound for this ideal gas compression comparison.

2 Model the practical single stage compressor

For pressure ratio r, calculate T_out,s = T_in r^[(gamma − 1)/gamma], T_out = T_in + (T_out,s − T_in)/eta_s, and compressor specific work w = cp(T_out − T_in). Compute shaft power m_dot w, motor electrical input m_dot w/eta_m, and entropy generation rate m_dot[cp ln(T_out/T_in) − R ln(r)]. Compute aftercooler heat rejection m_dot cp(T_out − 310 K). Check the 500 K discharge and 22 kW motor limits.

3 Design the two stage system

Evaluate stage one from p1 to p2 at T1 and stage two from p2 to p3 at T3. Sum both stage works; calculate motor input, intercooler rejection m_dot cp(T2 − T3), and aftercooler rejection m_dot cp(T4 − 310 K). Calculate entropy generation separately in each compressor. If modelling the coolers as heat rejection to the 293 K reservoir, use Sgen,cool = m_dot cp ln(T_out/T_in) + Q_out/Tc for each cooler and verify nonnegative values. Compare p2 = sqrt(p1 p3) with the optimum obtained numerically when T3 differs from T1.

4 Sweep, optimise and test robustness

Sweep p2 over 150–650 kPa, retaining only p1 < p2 < p3, and minimise total motor input subject to each stage discharge temperature no greater than 500 K and electrical input no greater than 22 kW. Repeat across delivery pressure 600–1,000 kPa and inlet temperature 288–308 K. Show where the system is infeasible. Do not extrapolate a solution outside the stated limits. Run at least 5,000 reproducible samples with uniform uncertainty over inlet pressure, inlet temperature, mass flow, eta_s, and eta_m; report the fraction satisfying all limits and state the independence assumption.

Python figures and validation

- Plot isothermal, reversible adiabatic and practical single stage specific work against delivery pressure; show separate curves for inlet temperatures 288, 298 and 308 K.
- Plot the single stage and both two stage discharge temperatures against delivery pressure, marking the 500 K limit. Plot two stage motor input against intermediate pressure and mark the optimum and 22 kW limit.
- Plot intercooler and aftercooler heat rejection against delivery pressure for the chosen design. Label every axis and unit; distinguish fixed inputs and swept parameters in figure captions.
- Use functions with documented units, input validation, and checks for energy balance, w_iso <= w_s <= w_actual for the common endpoints, nonnegative entropy generation, and agreement of numerical and analytic results for a reversible stage.

Suggested code structure

Implement separate functions for isothermal_work(T_in, p_in, p_out), compressor_stage(T_in, p_in, p_out, eta_s), cooler(T_in, T_out, m_dot, T_c), and system(p2, parameters). Return a named dictionary or dataclass with temperatures, specific work, power, cooler duties, entropy generation and feasibility flags. Use scipy.optimize.minimize_scalar with a bounded interval for p2 and verify the result against a pressure sweep. The functions should reject nonphysical temperatures, pressures, mass flow and efficiencies.

Illustrative Python figures

These figures illustrate the base model and the expected style of numerical results. They are generated by the sample script. Students should reproduce and extend the calculations, including uncertainty analysis, cooler entropy generation and the full feasibility map. The model curves are based on the stated teaching assumptions.

Figure 1 Work for three compression processes



Figure 2 Temperature constraints

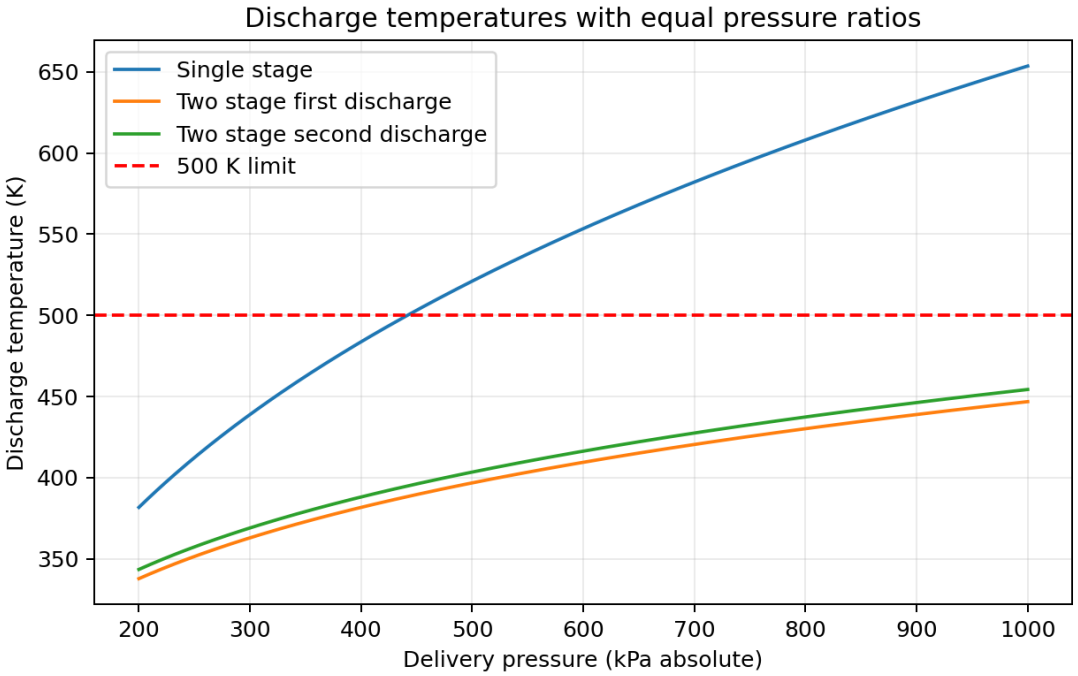
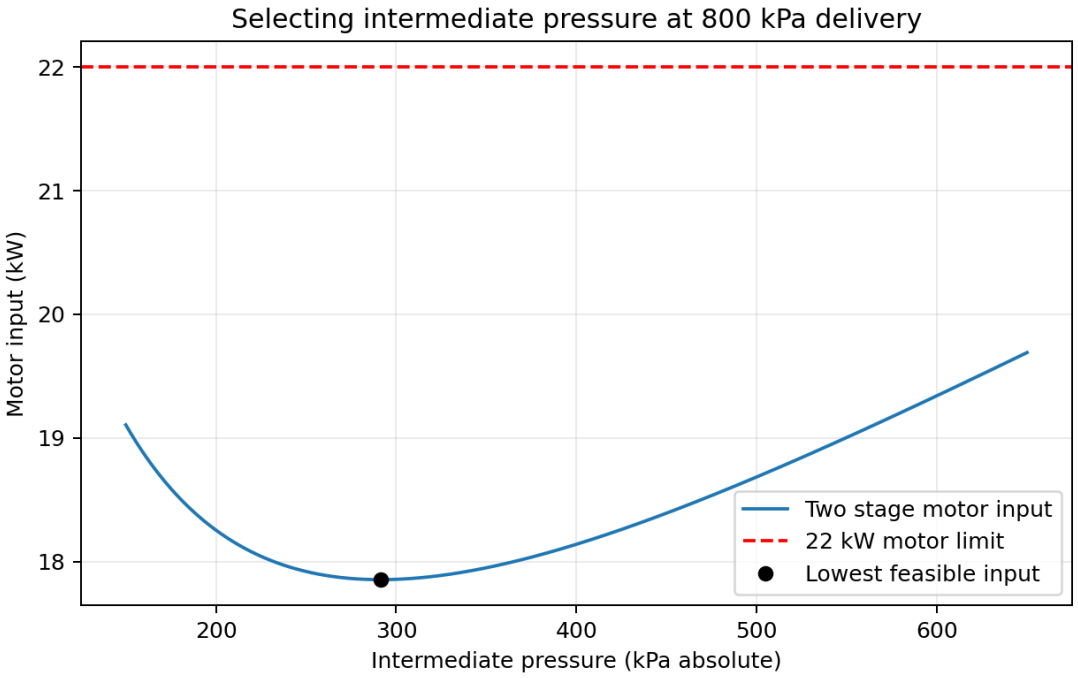


Figure 3 Intermediate pressure and motor input



Sample Python script

Save the code as isothermal_adiabatic_compressor_example.py. Install NumPy and Matplotlib, then run it with Python 3. It prints base case values and generates the three figures above. The pressure sweep approximates the constrained optimum; students should improve and verify the optimiser and complete the remaining project requirements.

```
"""Illustrative compressor analysis. Run with NumPy and Matplotlib installed."""
import numpy as np
import matplotlib.pyplot as plt


R, CP, GAMMA = 0.287, 1.005, 1.400  # kJ/(kg K), kJ/(kg K), dimensionless
P1, P3, T1, T3 = 100.0, 800.0, 298.0, 303.0  # kPa absolute, K
M_DOT, ETA_S, ETA_M = 0.060, 0.78, 0.90  # kg/s, fractions
T_LIMIT, MOTOR_LIMIT = 500.0, 22.0  # K, kW



def stage(t_in, p_in, p_out, eta_s=ETA_S):
    """Adiabatic compressor: outlet K, specific work kJ/kg, entropy kJ/(kg K)."""
    if np.any(np.asarray(t_in) <= 0) or np.any(np.asarray(p_in) <= 0) \
            or np.any(np.asarray(p_out) <= np.asarray(p_in)):
        raise ValueError('Require positive inlet state and increasing pressure.')
    if not 0 < eta_s <= 1:
        raise ValueError('Isentropic efficiency must be in (0, 1].')
    ratio = np.asarray(p_out) / p_in
    t_out_s = t_in * ratio ** ((GAMMA - 1) / GAMMA)
    t_out = t_in + (t_out_s - t_in) / eta_s
    work = CP * (t_out - t_in)
    delta_s = CP * np.log(t_out / t_in) - R * np.log(ratio)
    return t_out, work, delta_s



def two_stage(p_mid, p_final=P3):
    """Return stage discharges, motor input and cooler heat rejection rates."""
    if np.any((np.asarray(p_mid) <= P1) | (np.asarray(p_mid) >= p_final)):
        raise ValueError('Intermediate pressure must lie between inlet and outlet.')
    t2, w1, s1 = stage(T1, P1, p_mid)
    t4, w2, s2 = stage(T3, p_mid, p_final)
    motor_kw = M_DOT * (w1 + w2) / ETA_M
    intercooler_kw = M_DOT * CP * (t2 - T3)
    aftercooler_kw = M_DOT * CP * (t4 - 310.0)
    return t2, t4, motor_kw, intercooler_kw, aftercooler_kw, s1, s2


# Base case and feasible intermediate pressure selected from a fine sweep.
p_mid = np.linspace(150.0, 650.0, 2001)
t2, t4, motor, q_inter, q_after, s1, s2 = two_stage(p_mid)
feasible = (t2 <= T_LIMIT) & (t4 <= T_LIMIT) & (motor <= MOTOR_LIMIT)
if not feasible.any():
    raise RuntimeError('No feasible two stage design in the specified interval.')
best = np.flatnonzero(feasible)[np.argmin(motor[feasible])]
single_t, single_w, single_s = stage(T1, P1, P3)
print(f'Single stage: {single_t:.1f} K, {M_DOT*single_w/ETA_M:.2f} kW motor')
print(f'Two stages: p_mid={p_mid[best]:.1f} kPa; '
      f'T2={t2[best]:.1f} K; T4={t4[best]:.1f} K; '
      f'motor={motor[best]:.2f} kW')
print(f'Cooler duties: {q_inter[best]:.2f} and {q_after[best]:.2f} kW')
assert single_s >= -1e-10 and s1[best] >= -1e-10 and s2[best] >= -1e-10
assert np.isclose(CP*(single_t-T1), single_w)


# Figure 1: comparison of ideal and practical single stage work.
p_final = np.linspace(200.0, 1000.0, 300)
fig, ax = plt.subplots(figsize=(7, 4.5))
for t_in in (288.0, 298.0, 308.0):
    ratio = p_final / P1
    iso = R*t_in*np.log(ratio)
    isentropic = stage(t_in, P1, p_final, 1.0)[1]
    actual = stage(t_in, P1, p_final, ETA_S)[1]
    for y, label, ls in [(iso, 'Isothermal', ':'),
                         (isentropic, 'Isentropic', '--'),
                         (actual, 'Actual adiabatic', '-')]:
        ax.plot(p_final, y, ls, label=f'{label}, {t_in:.0f} K')
ax.set(xlabel='Delivery pressure (kPa absolute)',
       ylabel='Specific compressor work (kJ/kg)',
       title='Work required for single stage compression')
ax.grid(alpha=.25); ax.legend(ncol=2, fontsize=7); fig.tight_layout()
fig.savefig('compression_work_lines.png', dpi=180)


# Figure 2: discharge temperature of single and two stage arrangements.
single = stage(T1, P1, p_final)[0]
p_equal = np.sqrt(P1*p_final)
stage1 = stage(T1, P1, p_equal)[0]
stage2 = stage(T3, p_equal, p_final)[0]
fig, ax = plt.subplots(figsize=(7, 4.5))
```

```
ax.plot(p_final, single, label='Single stage')
ax.plot(p_final, stage1, label='Two stage first discharge')
ax.plot(p_final, stage2, label='Two stage second discharge')
ax.axhline(T_LIMIT, color='red', ls='--', label='500 K limit')
ax.set(xlabel='Delivery pressure (kPa absolute)', ylabel='Discharge temperature (K)',
       title='Discharge temperatures with equal pressure ratios')
ax.grid(alpha=.25); ax.legend(); fig.tight_layout()
fig.savefig('compressor_temperature_lines.png', dpi=180)


# Figure 3: intermediate pressure design at the 800 kPa delivery point.
fig, ax = plt.subplots(figsize=(7, 4.5))
ax.plot(p_mid, motor, label='Two stage motor input')
ax.axhline(MOTOR_LIMIT, color='red', ls='--', label='22 kW motor limit')
ax.plot(p_mid[best], motor[best], 'ko', label='Lowest feasible input')
ax.set(xlabel='Intermediate pressure (kPa absolute)', ylabel='Motor input (kW)',
       title='Selecting intermediate pressure at 800 kPa delivery')
ax.grid(alpha=.25); ax.legend(); fig.tight_layout()
fig.savefig('intermediate_pressure_lines.png', dpi=180)
plt.close('all')
```